Faculdade de Design,
Tecnologia e Comunicação
Universidade Europeia

03 – Supervised Learning

Automatic Applied Learning

Fernando Marson

fernando.marson@universidadeeuropeia.pt

Supervised Learning

- Supervised Learning is a Machine Learning approach in which a model learns from examples with known correct answers.

- Each example contains:
  - **Input features**: information used by the model;
  - **Class label**: the correct category.

© Copyright European University. All rights reserved

Model-Based vs. Instance-Based Learning

- Supervised learning algorithms **can learn in two main ways**:
  - **Model-Based**: by **building a model** from the training data;
  - **Instanced-based**: by directly **using stored training examples**.

© Copyright European University. All rights reserved

Model-Based Learning

- Model-based algorithms **use the training data to build a model**.

- **The model**
  - Learns patterns from the data;
  - Contains parameters adjusted during training;
  - Is used to make predictions about new examples.

- **Main idea**
  - The algorithm learns **a general rule** that **represents the data**.

© Copyright European University. All rights reserved

Multinomial Naive Bayes

- **Multinomial Naive Bayes** is a **Model-Based Learning** algorithm commonly used for text classification.
- It learns patterns from the **frequency of features associated with each class**.

- **How it works**
  1. Convert the input data into numerical features;
  2. Estimate the probability of each feature within each class;
  3. Calculate the probability of each possible class;
  4. Predict the class with the highest probability.

© Copyright European University. All rights reserved

Multinomial Naive Bayes

- **Multinomial Naive Bayes** is a **Model-Based Learning** algorithm commonly used for text classification.
- It learns patterns from the **frequency of features associated with each class**.

- **How it works**
  1. Convert the input data into numerical features;
  2. Estimate the probability of each feature within each class;
  3. Calculate the probability of each possible class;
  4. Predict the class with the highest probability.

© Copyright European University. All rights reserved

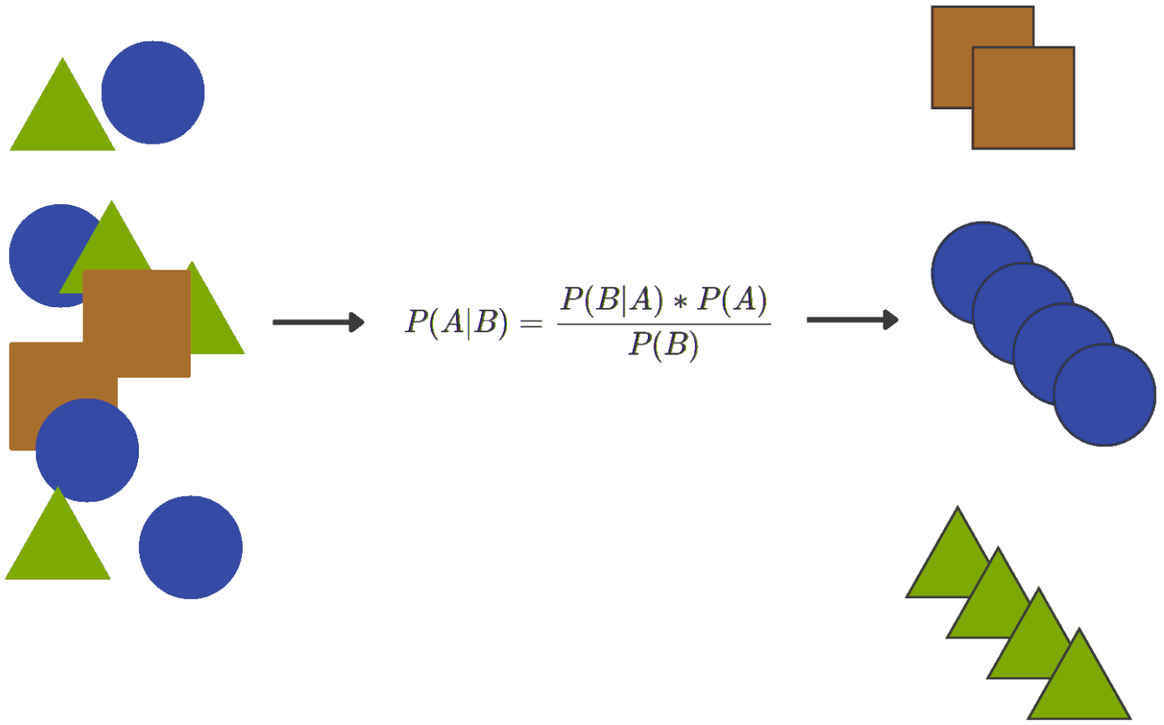
Multinomial Naive Bayes

- **Main assumption**
  - The algorithm assumes that **the features contribute independently to the prediction**, given the class.
  - This **assumption is often simplified** in practice, but the method can still perform well.

- **Main idea**
  - Learn class probabilities from the training data and use them to classify new examples.

© Copyright European University. All rights reserved

Multinomial Naive Bayes



© Copyright European University. All rights reserved

Support Vector Machine

- Support Vector Machine (SVM) is a **Model-Based Learning** algorithm that **learns a boundary between classes**.

- **How it works**
  - Represent each example as a feature vector;
  - Search for a decision boundary that separates the classes;
  - Select the boundary with the largest margin;
  - Use the learned boundary to classify new examples.

- **After the model has been built**
  - The learned parameters are sufficient to make predictions;
  - The training data can be discarded.

© Copyright European University. All rights reserved

Support Vector Machine

- For a linear SVM, the decision function can be written as:

$$f(x) = w^{*T}x + b^*$$

- **Where**
  - x represents the **input features**;
  - w^* contains the **learned weights**;
  - b^* is the **learned bias**.

- The prediction depends on the side of the boundary on which the example lies:

$$f(x) \geq 0 \Rightarrow \text{Class } 1 \; f(x) < 0 \Rightarrow \text{Class } 0$$

© Copyright European University. All rights reserved

Support Vector Machine



© Copyright European University. All rights reserved

Instance-Based Learning

- Instance-based algorithms **use the examples in the dataset as their reference**.
- Instead of building a general rule, these algorithms:
  - Store the training data;
  - Compare new examples with known examples;
  - Make predictions based on similarity or proximity.

- **Main idea**
  - The algorithm remembers the examples and uses the most similar ones to make a decision.

© Copyright European University. All rights reserved

k-Nearest Neighbors

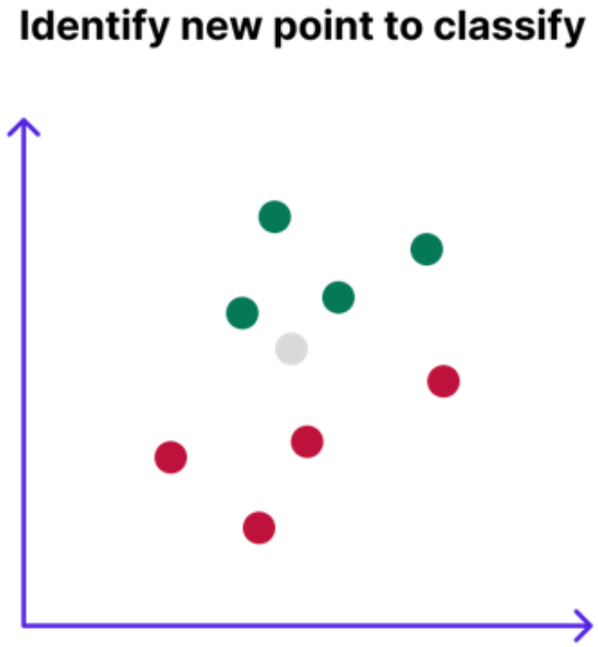
- k-Nearest Neighbors (kNN) is an **instance-based algorithm**.

- To classify a new observation, the algorithm:
  - Receives the new example;
  - Calculates its distance from the training examples;
  - Selects the $k$nearest examples;
  - Examines their classes;
  - Assigns the most frequent class.

© Copyright European University. All rights reserved

k-Nearest Neighbors

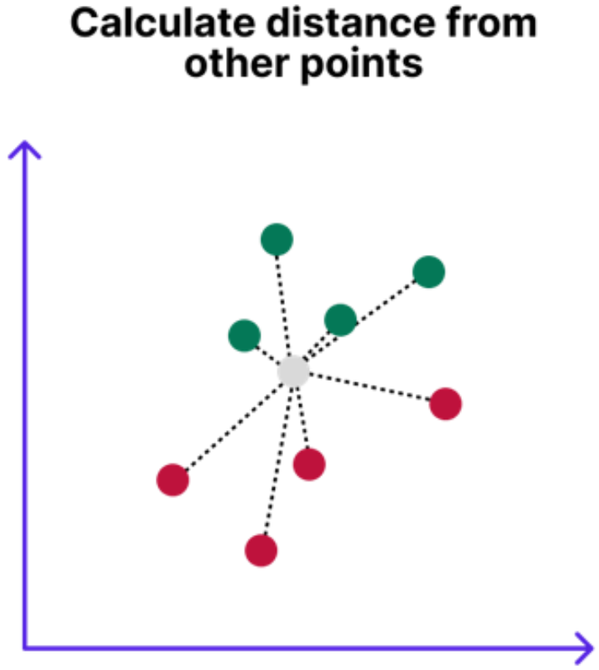
- Suppose we want to classify a new example.



© Copyright European University. All rights reserved

k-Nearest Neighbors

- Suppose we want to classify a new example.
- Among the k=3 nearest neighbors:
  - 2 belong to Class A;
  - 1 belong to Class B.

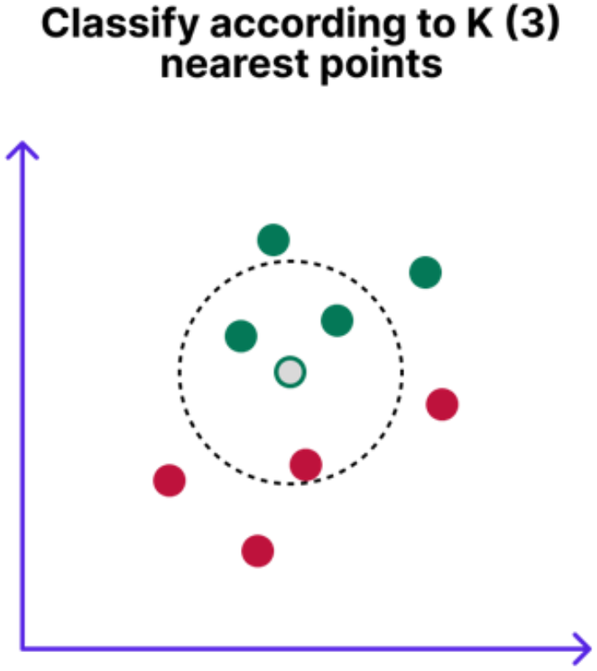


© Copyright European University. All rights reserved

k-Nearest Neighbors

- Suppose we want to classify a new example.
- Among the k=**3** nearest neighbors:
  - 2 belong to Class A;
  - 1 belong to Class B.
- The prediction will be

  Predicted class = **Class A**
- This happens because **Class A** is the **most frequent class among the selected neighbors**.



© Copyright European University. All rights reserved

Comparing Supervised Learning Techniques

- **Different ways to learn** from labeled data.

Characteristic	Model-Based Learning	Instance-Based Learning
Learning process	Builds a model	Stores the examples
Representation	Learned parameters	Training dataset
Prediction	Uses the model	Compares similar examples
Training data after training	Can be discarded	Must usually be kept
Example	SVM	kNN

© Copyright European University. All rights reserved

Comparing Supervised Learning Techniques

- **Supervised Learning** techniques **differ in how they learn** from the training data and **make predictions**.

- **Model-Based Learning**
  - learns parameters, probabilities, rules, or decision boundaries.

- **Instance-Based Learning**
  - keeps the training examples and uses similarity to make predictions.

© Copyright European University. All rights reserved

Comparing Supervised Learning Techniques

Technique	Learning approach	Main idea	Typical use
Multinomial Naive Bayes	Model-Based	Learns class probabilities from feature frequencies	Text classification
Support Vector Machine	Model-Based	Learns a decision boundary with a large margin	Classification
Decision Tree	Model-Based	Learns a sequence of decision rules	Classification and regression
Logistic Regression	Model-Based	Learns probabilities using a mathematical function	Binary classification
k-Nearest Neighbors	Instance-Based	Compares a new example with nearby training examples	Classification and regression
Random Forest	Model-Based	Combines predictions from several decision trees	Classification and regression

© Copyright European University. All rights reserved

Supervised Learning - SMS Spam Collection

- https://archive.ics.uci.edu/dataset/228/sms+spam+collection
- In the **SMS Spam Collection** dataset, each example contains:
  - **Input**: the text of an SMS message;
  - **Class label**:
    - **spam** = unwanted or fraudulent message;
    - **ham** = legitimate message, or not spam.

- **During training**, the model **learns patterns** associated with spam and legitimate messages.
- **After training**, it **uses these patterns** to classify new messages.

© Copyright European University. All rights reserved

SMS Spam Collection

ham Go until jurong point, crazy.. Available only in bugis n great world la e buffet... Cine there got amore wat...

ham Ok lar... Joking wif u oni...

spam Free entry in 2 a wkly comp to win FA Cup final tkts 21st May 2005. Text FA to 87121 to receive entry question(std txt rate)T&C's apply 08452810075over18's

ham U dun say so early hor... U c already then say...

ham Nah I don't think he goes to usf, he lives around here though

© Copyright European University. All rights reserved

Python Packages

1. Open your VENV
2. Install all dependencies

```
pip install pandas scikit-learn matplotlib
```

© Copyright European University. All rights reserved

Workflow

Load the dataset

↓

Split the data into training and test sets

↓

Convert messages into numerical features

↓

Train the classifier

↓

Make predictions

↓

Evaluate the predictions

© Copyright European University. All rights reserved

Load the dataset with Python

```
import pandas as pd

df = pd.read_csv(
    "SMSSpamCollection",
    sep="\t",
    header=None,
    names=["label", "message"]
)

print(df.head())
print()
print(df.shape)
print()
print(df["label"].value_counts())
```

© Copyright European University. All rights reserved

Prepare the labels

```
df["label"] = df["label"].map({
    "ham": 0,
    "spam": 1
})

print(df.head())
```

© Copyright European University. All rights reserved

Separate inputs and labels

```
X = df["message"]
y = df["label"]
```

© Copyright European University. All rights reserved

Split the dataset into training and test data

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.25,
    random_state=42,
    stratify=y
)

print("Training messages:", len(X_train))
print("Test messages:", len(X_test))
```

© Copyright European University. All rights reserved

Convert text into numerical features

```
from sklearn.feature_extraction.text import TfidfVectorizer

vectorizer = TfidfVectorizer(
    lowercase=True,
    stop_words="english"
)

X_train_vectorized = vectorizer.fit_transform(X_train)
X_test_vectorized = vectorizer.transform(X_test)

print("Training matrix shape:", X_train_vectorized.shape)
print("Test matrix shape:", X_test_vectorized.shape)
```

© Copyright European University. All rights reserved

Common evaluation functions

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    confusion_matrix,
    ConfusionMatrixDisplay,
    classification_report
)

import matplotlib.pyplot as plt
```

© Copyright European University. All rights reserved

Common evaluation functions

```python
def evaluate_model(model_name, y_true, y_pred):
    cm = confusion_matrix(y_true, y_pred)

    tn, fp, fn, tp = cm.ravel()

    accuracy = accuracy_score(y_true, y_pred)

    precision = precision_score(
        y_true,
        y_pred,
        zero_division=0
    )

    recall = recall_score(
        y_true,
        y_pred,
        zero_division=0
    )

    specificity = tn / (tn + fp)

    f1 = f1_score(
        y_true,
        y_pred,
        zero_division=0
    )
```

```python
print(f"\n{model_name}")
print("-" * len(model_name))
print(f"Accuracy:    {accuracy:.3f}")
print(f"Precision:   {precision:.3f}")
print(f"Recall:      {recall:.3f}")
print(f"Specificity: {specificity:.3f}")
print(f"F1 score:    {f1:.3f}")

print("\nClassification report:")
print(
    classification_report(
        y_true,
        y_pred,
        target_names=["Not spam", "Spam"],
        zero_division=0
    )
)

display = ConfusionMatrixDisplay(
    confusion_matrix=cm,
    display_labels=["Not spam", "Spam"]
)

display.plot(cmap="Blues")
plt.title(f"{model_name} — Confusion Matrix")
plt.show()
```

```python
return {
    "model": model_name,
    "accuracy": accuracy,
    "precision": precision,
    "recall": recall,
    "specificity": specificity,
    "f1": f1
}
```

© Copyright European University. All rights reserved

Train and Prediction

© Copyright European University. All rights reserved

Multinomial Naive Bayes

```
from sklearn.naive_bayes import MultinomialNB

naive_bayes_model = MultinomialNB()

naive_bayes_model.fit(
    X_train_vectorized,
    y_train
)

nb_predictions = naive_bayes_model.predict(
    X_test_vectorized
)
```

```
nb_results = evaluate_model(
    "Multinomial Naive Bayes",
    y_test,
    nb_predictions
)
```

© Copyright European University. All rights reserved

Support Vector Machine

```
from sklearn.svm import LinearSVC

svm_model = LinearSVC(
    random_state=42
)

svm_model.fit(
    X_train_vectorized,
    y_train
)

svm_predictions = svm_model.predict(
    X_test_vectorized
)
```

```
svm_results = evaluate_model(
    "Support Vector Machine",
    y_test,
    svm_predictions
)
```

© Copyright European University. All rights reserved

k-Nearest Neighbors

```
from sklearn.neighbors import KNeighborsClassifier

knn_model = KNeighborsClassifier(
    n_neighbors=5,
    metric="cosine",
    algorithm="brute"
)

knn_model.fit(
    X_train_vectorized,
    y_train
)

knn_predictions = knn_model.predict(
    X_test_vectorized
)
```

```
knn_results = evaluate_model(
    "k-Nearest Neighbors",
    y_test,
    knn_predictions
)
```

© Copyright European University. All rights reserved

Compare the three models

```
results = pd.DataFrame([
    nb_results,
    svm_results,
    knn_results
])

print(results)
```

Show only the main metrics

```
 print(
     results[
         [
             "model",
             "accuracy",
             "precision",
             "recall",
             "specificity",
             "f1"
         ]
     ].round(3)
 )
```

© Copyright European University. All rights reserved

Predict new messages with Python

```
new_messages = [
    "Congratulations! You have won a free prize. Call now!",
    "Can we meet after class to discuss the project?",
    "Urgent! Claim your reward before it expires!",
    "I will send you the notes this evening."
]

new_messages_vectorized = vectorizer.transform(
    new_messages
)
```

```
nb_new_predictions = naive_bayes_model.predict(
    new_messages_vectorized
)

svm_new_predictions = svm_model.predict(
    new_messages_vectorized
)

knn_new_predictions = knn_model.predict(
    new_messages_vectorized
)
```

© Copyright European University. All rights reserved

Predict new messages with Python

```
def label_name(value):
    return "Spam" if value == 1 else "Not spam"


for i, message in enumerate(new_messages):
    print()
    print("Message:", message)
    print(
        "Naive Bayes:",
        label_name(nb_new_predictions[i])
    )
    print(
        "SVM:",
        label_name(svm_new_predictions[i])
    )
    print(
        "kNN:",
        label_name(knn_new_predictions[i])
    )
```

© Copyright European University. All rights reserved